Developer's Guide

aka: How we do things here (mostly)

Define module `Guidelines`

- Add `guidelines` folder in `./app/domain/`
- Add module file `guidelines.rb` in `./app/domain/` folder
- The `./app/domain/guidelines.rb` file should look like:

```
module Guidelines
  def self.subscriptions
    [
    ].map(&:subscriptions).reduce(&:merge)
  end
end
```

- register `Guidelines` module subscriptions by adding `Guidelines.subscriptions` to the `handlers` method in `./lib/application_subscriptions.rb`:

```
  def handlers
    top_level_subscriptions
      .merge(SomeOtherModule.subscriptions)
      ...
      .merge(Guidelines.subscriptions)
  end
```

Define Domain Events

- in the module file add domain events that will be published by this module. Make sure the schema is defined in `SCHEMA` constant and the list of stream names where a fact should be published to are defined in an instance method `stream_names`:

```
class RfcIssued < Fact
  SCHEMA = {
    tid: String,
    developer_id: String,
    number: String,
```

```
  }.freeze

  def stream_names
    ["RFC$#{data.fetch(:tid)}"]
  end
end
```

```
class RfcApprovedByDeveloper < Fact
  SCHEMA = {
    tid: String,
    developer_id: String,
  }.freeze

  def stream_names
    ["RFC$#{data.fetch(:tid)}", "Developer$#{data.fetch(:developer_id)}"]
  end
end
```

```
class RfcApproved < Fact
  SCHEMA = {
    tid: String,
  }.freeze

  def stream_names
    ["RFC$#{data.fetch(:tid)}"]
  end
end
```

As you might have noticed `RfcIssued` has more than one streams defined. In this case the domain event will be published in the first (main) stream and will be linked to other streams from the list.

Read more: about defining domain events

Define Commands & command handlers

We use 2 kinds of commands. First is simple self executing command. We use it when there is no much dependencies required to execute the command. It is based on GOF's command pattern. Self executing commands are defined in separate files in domain's module folder. i.e. for approve action it will be `./app/domain/guidelines/approve_by_developer.rb`:

```
module Guidelines
  class ApproveByDeveloper
    include Command::Executable
    attribute :tid, String
    attribute :developer_id, String
```

```
    validates :tid, :developer_id, presence: true

    def call
      rfc = ObjectRepository.find(tid)
      rfc.approve_by!(developer_id)
    end
  end
end
```

There are commands which execution is more complex and requires some external dependencies or have some common parts used in execution method. Usually this commands are handled by service object (command handler). This command handler is defined in module's domain folder, usually as `./app/domain/service.rb`

```
module Guidelines
  class Service
    include Command::Handler

    handles IssueRequestForComment, :issue_rfc
    handles ApproveRequestForComment, :approve_rfc

    def initialize(rfc_number_generator = SomeNumberGenrator.new)
      @rfc_number_generator = rfc_number_generator
    end

    private
    attr_reader :rfc_number_generator

    def issue_rfc(cmd)
      rfc = RequestForComment.create!(
        number: rfc_number_generator.call,
        description: cmd.description,
        author_id: cmd.developer_id,
      )
      event_store.publish(RfcIssued.strict(data: {
        tid: rfc.tid,
        developer_id: rfc.developer_id,
        number: rfc.number,
      }))
    end

    def approve_rfc(cmd)
      rfc = ObjectRepository.find(cmd.tid)
      rfc.approve!
    end
  end
end
```

Executable commands are self-registering themselves in the application command bus. Command handlers for non-executable commands should be registered in the application command bus

`register_commands` method. Both kind of commands should be triggered only by sending commands to the command bus. This ensures that instrumentation is properly setup and the command(s) execution is wrapped in a database transaction.

We do not call commands dirrectly here.

Command bus (or a set of command bus classes connected by chain-of-responsibility pattern) are responsible for handling such concerns as:

- transactions
- instrumentation
- setting up causation & correlation ids of command is executed in context of some domain event handler (in that case correllation id will be the same as handled comain event correlation id and causation id will be the id of the domain event that caused command execution)

Details of `command` & `command handler` patterns implementation could be found in ./lib/command.rb and `command bus` implementations could be find in ./lib/command/ folder.

Define business domain object

- define model for new domain object

```
module Guidelines
  class RequestForComment < ApplicationRecord
    include EventStoreInjector
    has_typeid :rfc
    has_many :approvals

    def approve_by!(developer_id)
      approvals.create!(developer_id: developer_id)
      event_store.publish(
        RfcApprovedByDeveloper.strict(
          data: {
            tid: tid,
            developer_id: developer_id
          }
        )
      )
    end

    def approve!
      event_store.publish(
        RfcApprovedByDeveloper.strict(data: { tid: tid })
      )
    end
```

```
  end
end
```

```
module Guidelines
  class Approval < ApplicationRecord
    has_one :developer
  end
end
```

- register type ID for newly created domain object in `ObjectRepository` (file: `config/initializers/object_repository.rb`):

```
Rails.configuration.to_prepare do
  SomeOtherModule::DomainObject
  ...
  Guidelines::RequestForComment
end
```

Proces managers & event handlers

For complex or long lived (in time) processes we use `process managers` to track process state and trigger an action when needed. The proces manager is function taking events and maybe dispatching a command (`->(events) -> command`). This could be part of the domain module if it is internal process i.e. approval of RFC when it is approved by sufficient number of developers:

```
module Guidelines
  class RfcApprovalProcess
    include Handler.async(queue: "low")
    subscribes_to RfcApprovedByDeveloper

    def initializer(approval_policy = JustEnoughApprovalPolicy.new)
      @approval_policy = approval_policy
    end

    def call(fact)
      tid = fact.data.fetch(:tid)
      approvals = event_store.read.stream("RFC$#{tid}").count
      if approval_policy.satisfied?(approvals)
        command_bus.call(Guidelines::ApproveRequestForComment.new(tid: tid))
      end
    end
  end
end
```

The approval policy might be implemented as `HOWITISCALLED`:

```
class JustEnoughApprovalPolicy
  def satisfied?(value)
    value >= 2
  end
end
```

You need to remember to register `RfcApprovalProcess.subscriptions` in the `subscriptions` method `Guidelines` module and register `Guidelines.subscriptions` in `ApplicationSubscriptions`.

Event handlers or process managers (when we need more than single domain event to determine process state) might be used to orchestrate collaboration between different modules. It should be created in `./app/integrators` folder. I.e. handler that adds refactor task in ClickUp when RFC is approved could be implemented (in. `./app/integrators/guidelines_integrator.rb`):

```
module GuidelinesIntergators
  class ScheduleCodeRefactorWhenRfcApproved
    include Handler.async(queue: "low")
    subscribes_to Guidelines::RfcApproved

    def call(fact)
      rfc = ObjectRepository.find(fact.data.fetch(:tid))
      command_bus.call(ClickUp::AddNewTaskToTheBacklog.new(**build_task(rfc)))
    end

    private

    def build_task(rfc)
      # ... do it here
    end
  end

  def self.subscriptions
    [
      ScheduleCodeRefactorWhenRfcApproved.subscriptions,
    ].reduce(&:merge)
  end
end
```

You need to remember to register `GuidelinesIntergators.subscriptions` in the `handlers` method in `./lib/application_subscriptions.rb`.

Read models

Read models are powerfull tool that allow us to quickly fetch complex to define data that are prepared in a form ready to use and easy to read asynchronously.

Read models usually are build when some domain event is published. There could be different kinds of read models - the ones used by domain module to get some summary data from other modules that are needed to execute the module's logic or the application read models used mainly to present data to the user.

The application read model could be simple event handler:

```
module ReadModels
  class NumberOfRfcIssuedByDeveloper
    include Handler.async(queue: "low")
    subscribes_to Guidelines::RfcIssued

    class Count < ApplicationRecord
      self.table = "number_of_rfc_issued_by_developer"
    end

    def call(fact)
      developer_id = fact.data.fetch(:developer_id)
      ApplicationRecord.transaction do
        Count.find_or_initialize_by(developer_id: developer_id).lock!.tap do
|record|
          record.value += 1
          reco.save!
        end
      end
    end
  end
end
```

The read models could be rebuild at any time based on domain events. It is usefull to have domain events for a read model linked into single stream to be able to quickly select the facts used to rebuild a read model. I.e. script to rebuild count of RFC issues for a single developer could look like:

```
load("script/likeasir.rb")
developer_id = ARGV[0]

facts = event_store.read.stream("RFCIssuedByDeveloper$#{developer_id}")
progress = build_progress_bar(facts.count, "Count RFC issued")
ReadModels::NumberOfRfcIssuedByDeveloper.where(developer_id:
developer_id).delete_all
count = ReadModels::NumberOfRfcIssuedByDeveloper.new
facts.each do |fact|
  count.call(fact)
  progress.increment
end
```

Rails app

Most of the Ruby On Rails features we use as-is. The main difference is we try to keep Rails controllers as thin as possible. No logic there. Just parse params, create a command and send it to the command bus.

```
class RequestForCommentApprovalsController < ApiController
  def create
    command_bus.call(
      Guidelines::ApproveByDeveloper.new(
        tid: params[:id],
        developer_id: params[:developer_id]
      ))
  end
end
```

Incomming Webhooks

Almost all incomming webhooks should be handled asyncronously. Not only we reduce the time needed for webhook request, but also we store the payload (for audit log) and allow us to use our resilience practices to handle the webhook in case of some issues.

Typical webhook controller & processing job:

```
class SomeIncommingWebhookComntroller < ApplicationController
 def create
  payload = JSON.parse(request.body.read, symbolize_names: true)
  IncomingWebhook.create!(service: "some_service", data: payload.to_h).tap do
|webhook|
    SomeServiceWebhookJob.perform_async(webhook.id)
  end
  rescue SomeService::Error => exc
    ErrorReporting.notify(exc)
    head(:bad_request)
  end
```

```
class SomeServiceWebhookJob
  Error = Class.new(StandardError)
  include Sidekiq::Worker
  sidekiq_options queue: "critical"

  def perform(webhook_id)
    IncomingWebhook.find(webhook_id).tap do |webhook|
      return if webhook.handled?
      handled = call(webhook.data.with_indifferent_access)
      webhook.mark_as_handled(Time.zone.now, handled)
    end
  rescue
    raise Error
```

```
  end

  private

  def call(data)
    # ... handle logic here
  end
end
```

3rd party integration

To implement an integration with some 3rd party system we build an ACL module in `./app/integrations` folder. I.e. to integrate with ClickUp API we will add `./app/integrations/click_up` folder and `./app/integrations/click_up.rb` file.

In the `click_up.rb` we will define:

- domain events published by the integration
- define commands to twigger actions on ClickUp API
- define our own error classes to not allow 3rd party errors to bubble up

Usually we add the API adapter file and setup it for each environment by setting `Rails.configuration` object like:

```
Rails.configuration.to_prepare do
  # ...
  Rails.configuration.clickup = ClickUp::Adapter.new(
    api_key: Rails.application.credentials.dig(:click_up, :api_key)
  )
  # ...
end
```

For test environemnt we usually build a fake adapter to avoid stubbing real API request. We setup it in the `./config/environments/test.rb`.